PanelBatch: faster trajectory writes with unchanged study outcomes

brodyga45 | AI-assisted implementation and report | Private local draft, not submitted

Abstract

We evaluate an opt-in trajectory writer for SocioVerse2. The current writer already commits one transaction per step, but passes individual parameter rows through executemany. PanelBatch constructs a typed Arrow table and performs one INSERT SELECT per panel batch. The contribution is a compatible implementation and an explicit preservation experiment, not a new database algorithm. All experiments use synthetic agents and deterministic decisions; there are no live language-model calls.

For 2,048 agents over six steps and three seeds, the median full simulator run decreased from 3.851 to 0.473 seconds (8.14x). Ordered panel values, metrics, events, portable exports and a replayed branch matched. A separate fresh-process check found a smaller runtime advantage and higher peak resident memory. These results support a local storage bottleneck improvement, not a claim about faster model inference or valid social predictions.

1. Research question and practical scope

Longitudinal simulation persists one row per agent per time step. Researchers comparing interventions or repeating seeds can therefore produce large panel datasets even when their decision rule is inexpensive. Can replacing the panel insertion mechanism reduce runtime while preserving the observable research record and failure semantics? The unit of evaluation is the runtime and its generated data. The study does not estimate actual opinions or intervention effects.

The population P consists of declared synthetic agents: 128-8,192 records per batch in the component experiment, and 256 or 2,048 agents in the full study. Environment E and behavior B in the full study are the official opinion_diffusion template and its scripted decision rule. We change persistence only. The baseline reference study, agent model, seed, horizon, events and decision sequence remain fixed between writers.

Contribution and context

SocioVerse2 describes longitudinal and controllable research loops [1]. Preserving the resulting history matters for comparisons between versions and counterfactual branches. DuckDB already supports SQL over Arrow objects [2]; our contribution applies that established mechanism through the existing store registry and tests its implications for the current framework. Competitive novelty and eligibility remain for the organizers to assess.

2. Implementation and experimental design

The upstream dependency is pinned to commit 9c3ac774816511c121184ca11f539dc398ba1544. No upstream source or reference artifacts were edited. The new class inherits DuckDbTrajectoryStore and registers as panel_batch.arrow. The official builder selects it via SimulationConfig.store_ref; no patch to the engine is required.

Boundary	Baseline	PanelBatch
Serialization	JSON text for state and action payload	Same JSON serialization and six-column schema
Insertion	Parameter-row executemany within one transaction	Typed Arrow relation, one INSERT SELECT within one transaction
Failure	Rollback failed transaction	Rollback failed transaction; unregister temporary relation
Other records	Original events, messages, metrics and Parquet code	Inherited unchanged

Component experiment

Each case preconstructs four batches, including an initial state with null action fields. Values include Unicode, nested objects, arrays and nulls. Timed work includes serialization and database writes; it excludes row construction, connection/schema setup and finalization. Each writer runs three times, alternating execution order in the middle repetition. The check compares a hash of all sorted database values, not merely row counts. No simulated LLM latency is inserted.

Agents	Total rows	Baseline median, s	Bulk median, s	Ratio
128	512	0.1137	0.0095	11.94x
1,024	4,096	0.7830	0.0271	28.89x
8,192	32,768	8.1620	0.2135	38.23x

These write-only ratios are intentionally separated from full-run ratios. They should not be presented as acceleration of an entire scientific workflow. Both writers retain a transaction per batch, so the result is not a comparison against an intentionally weakened per-row-commit baseline.

Failure experiment

A valid initial batch is followed by a two-row insertion whose second JSON value is malformed. Both writers raise a database error; a before/after fingerprint confirms that neither new row remains. A subsequent valid batch commits, demonstrating connection recovery rather than leaving a dangling transaction. This covers transaction failure, not power loss or operating-system crash durability.

3. Complete study and compatibility

The full study uses the unmodified official scripted opinion model for six steps. For every writer and seed (17, 42, 99), sim.run includes population setup, environment evolution, decisions, memory, panel/progress files, database writes, aggregate metrics and Parquet finalization. Result queries and equality checks occur after the timer. All panels contain N x 7 rows, counting the initial state. This is a systems workload over synthetic agents, not an empirical social-science finding.

Agents	Baseline runs, seconds	Bulk runs, seconds	Median ratio
256	2.284, 0.599, 0.583	0.767, 0.157, 0.138	3.81x
2,048	3.416, 4.375, 3.851	0.522, 0.467, 0.473	8.14x

The 256-agent result varies substantially across runs, including first-use initialization. We report all timings and do not fit a scaling law or confidence interval to three observations. A preliminary series measured 2.01x and 9.96x; those pilot timings remain in bulk-study-probe-result.json. This archived confirmation series retains every database, Parquet export and live trace. Another machine, process state, filesystem or dependency version may change the magnitude.

Preservation checks

Scenario	Observed result
Full study	Exact equality of sorted panel values, all aggregate metric rows and events for each seed.
Portable output	Each Parquet panel equals its database source. Both writers use original finalization.
Counterfactual replay	A parent history is inherited through step 3. Both child writers decide only at steps 4-6; child outputs match and inherited states/actions/metrics reproduce the parent prefix. The expected action_source change to replay is excluded only in the parent-prefix comparison.
Official interface	Four focused checks passed in 2.16 seconds after extracting the class into its own module and selecting it through the official registry.
Other store behavior	The actual upstream engine smoke and transaction/messages assertions were reused with the compatible store.

The pilot class was extracted into its own module without changing its algorithm. The archived series uses the official registry integration. It was run to retain full experimental data, not to select a better timing: both series are disclosed. The additional 128-agent recorded demonstration also retains typed study artifacts and complete trajectories; it is not used as a performance estimate.

4. Resource costs, limits and reproduction

A fresh process was launched for each writer at 2,048 agents and seed 42, using the registered store path. Peak resident memory includes imports, simulation and output validation; it is not a per-batch allocation measurement. CPU time covers simulation and validation. There was one observation per writer, so these values are a resource check rather than an independent statistical performance estimate.

Writer	sim.run, s	CPU incl. checks, s	Peak RSS, MB
baseline	4.147	4.882	160.1
bulk	0.852	0.765	179.5

This check found a runtime improvement with an approximately 19.5 MB higher peak footprint. The writer materializes columns and an Arrow table, so it is not a streaming or bounded-memory improvement. For memory-constrained or model-bound studies, the tradeoff may be unattractive. No conclusion is drawn about energy consumption or monetary profitability.

Limitations and unresolved validation

All measurements are from one macOS ARM64 host with possible competing workloads. Linux and independent judging are not yet verified. The experiment does not cover concurrent writers, crash recovery, very large JSON payloads, external data services or live language models. Component speedups cannot be multiplied into LLM-heavy studies: decision time may dominate. The inherited class depends on upstream private connection state, so future upstream changes require compatibility review. The work is not a general benchmark of Arrow versus all DuckDB ingestion methods.

Reproduction and assistance disclosure

The package contains original runner sources, the compatible store, pinned dependencies, raw timing JSON and focused semantic checks. With the pinned upstream in upstream/, create a Python 3.12 virtual environment, install requirements-probe.txt and run PANEL_BATCH_PYTHON=venv/bin/python ./reproduce.sh. memory_probe.py runs the separate fresh-process check. Temporary databases and live logs use the OS temporary directory. No API credentials, real-person data or paid service is needed. Upstream retains its Apache-2.0 license and NOTICE.

Implementation, experiment orchestration and this draft were AI-assisted. There were zero simulation LLM calls, no model identifiers to report and no external model API spend in these experiments. General development/research assistance and local computing costs are not measured here and are not asserted to be free. No human source-code review or independent replication is claimed.

References

[1] X. Zhang et al. SocioVerse2: A Longitudinal Dynamic Social Simulation Framework under a Human-AI Co-evolutionary Paradigm. arXiv:2609.24911, 2026. https://arxiv.org/abs/2609.24911

[2] DuckDB documentation. SQL on Apache Arrow, current 1.5 documentation, accessed 21 October 2026. https://duckdb.org/docs/current/guides/python/sql_on_arrow

[3] Official SocioVerse2 source, pinned commit listed above. https://github.com/sii-research/SocioVerse2